
🛠️ Mermaids.js: Diagrams as Code Practice

---

we are moving away from heavy modeling tools and focusing on **Diagrams as Code (DaC)** using **Mermaid.js**.

In modern DevOps and Agile environments, we want our documentation to live alongside our code. Mermaid.js allows you to render Markdown-inspired text into complex diagrams. This ensures your software design remains version-controlled and easy to update without needing a graphic designer.

Lab Setup: Getting Started

You have two primary ways to use Mermaid.js. For this practice, we will start with the Live Editor and move to an Integrated Development Environment (IDE).

Option A: The Mermaid Live Editor (Beginner)

1. Go to Mermaid.live.
2. This is a web-based GUI where you can type code on the left and see the diagram on the right.

Option B: VS Code Extension (Professional Standard)

1. Open **VS Code**.
2. Go to the **Extensions** view (**Ctrl+Shift+X**).
3. Search for and install "**Mermaid Editor**" or "**Markdown Preview Mermaid Support**".
4. Create a new file named `practice.md`.
5. Wrap your mermaid code in a code block like this:

````
```mermaid
[Your Code Here]
```
````

Level 1: Beginner – Class Diagrams

Class diagrams describe the **static structure** of a system. We will model a simple **Library Management System**.

Key Syntax:

- `classClassName`: Defines a class.
- `<|--`: Inheritance (Is-a).
- `*--`: Composition (Has-a).
- `+`: Public / `-`: Private.

Step 1: Define the Classes

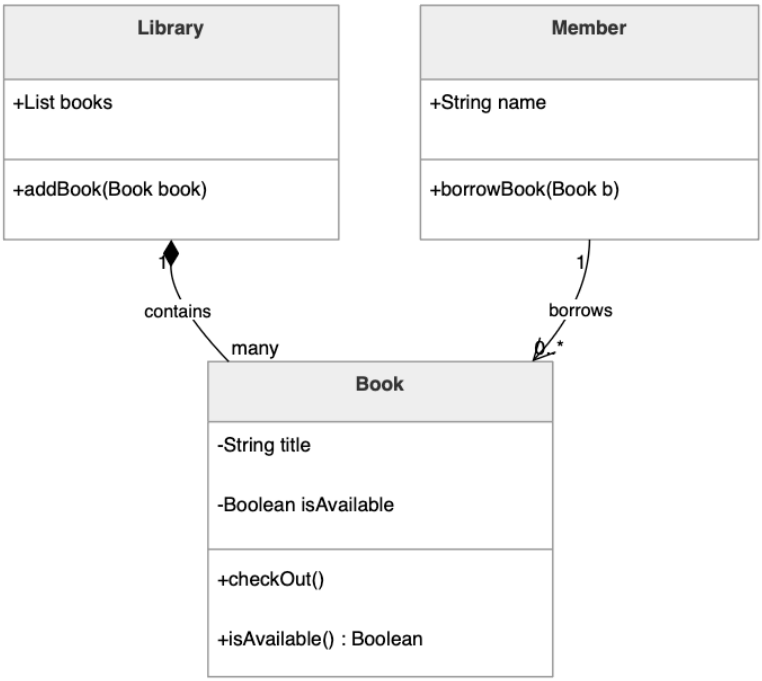
Copy this into your editor:

Code snippet

```
classDiagram
  class Book {
      -String title
      -Boolean isAvailable
      +checkOut()
      +isAvailable() Boolean
  }
  class Member {
      +String name
      +borrowBook(Book b)
  }

  class Library {
      +List books
      +addBook(Book book)
  }

  Library "1" *-- "many" Book : contains
  Member "1" --> "0..*" Book : borrows
```

- **Composition (`*--`):** The Library "owns" the books. If the library closes, the book collection (in this context) ceases to exist as a managed unit.
- **Multiplicity (`"1" *-- "many"`):** Indicates one library holds many books.

Class Diagram



Level 2: Intermediate – Sequence Diagrams

Sequence diagrams model the **dynamic behavior**—how objects interact over time. We will model the "**Checkout Process**".

Key Syntax:

- `participant`: Defines an object or actor.
- `->>`: Synchronous message.
- `-->>`: Response message.
- `alt/else`: Conditional logic (if/then).

Step 2: Modeling the Interaction

Add this below your class diagram:

Code snippet

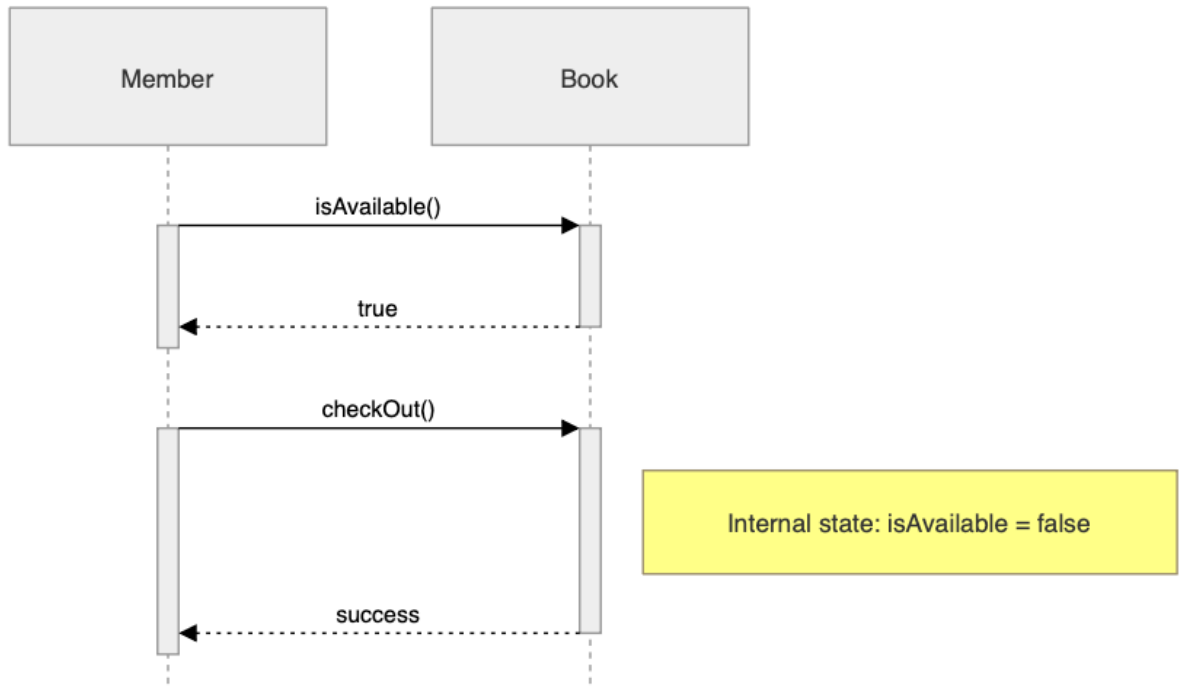
```
sequenceDiagram
    participant M as Member
    participant B as Book

    M->>B: isAvailable()
    activate M
    activate B
    B-->>M: true
```

```
deactivate B
deactivate M

M->>B: checkOut()
activate M
activate B
Note right of B: Internal state: isAvailable = false
B-->>M: success
deactivate B
deactivate M
```

Observation Task:

Notice the **Activation Boxes** (the vertical bars). You can add `activate S` and `deactivate S` to show exactly when a process is consuming memory or processing power.

Sequence Diagram



Level 3: Advanced – Software Design Integration

In a professional environment, we use these diagrams to explain complex architectures, such as **Microservices or API Gateways**.

📍 The Scenario: E-Commerce Payment Gateway

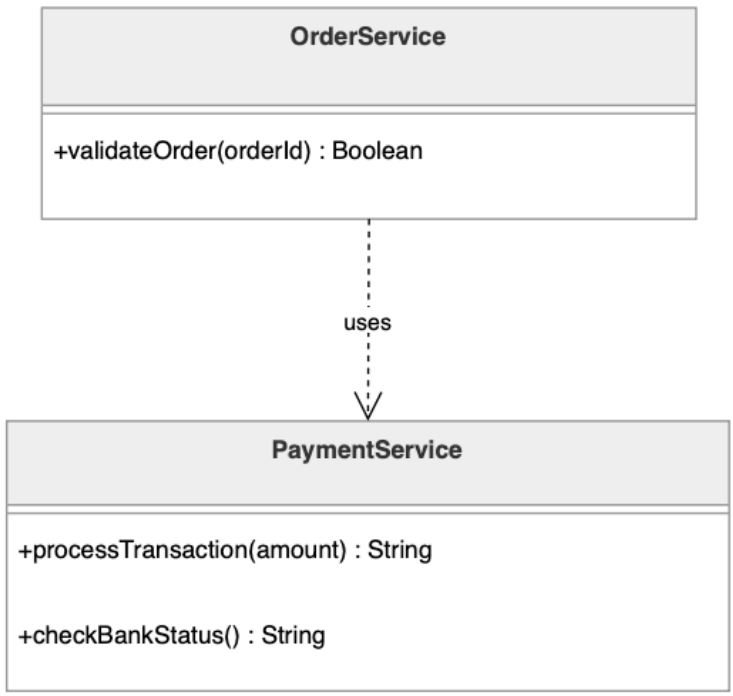
We need to design a system where a **User** pays for an order. The **Order Service** must talk to a **Payment Service**, which validates with an **External Bank API**.

Advanced Class Diagram with Generics and Annotations:

Code snippet

```
classDiagram
    class OrderService {
        +validateOrder(orderId) Boolean
    }
    class PaymentService {
        +processTransaction(amount) String
        +checkBankStatus() String
    }
    OrderService ..> PaymentService : uses
```

Class Diagram

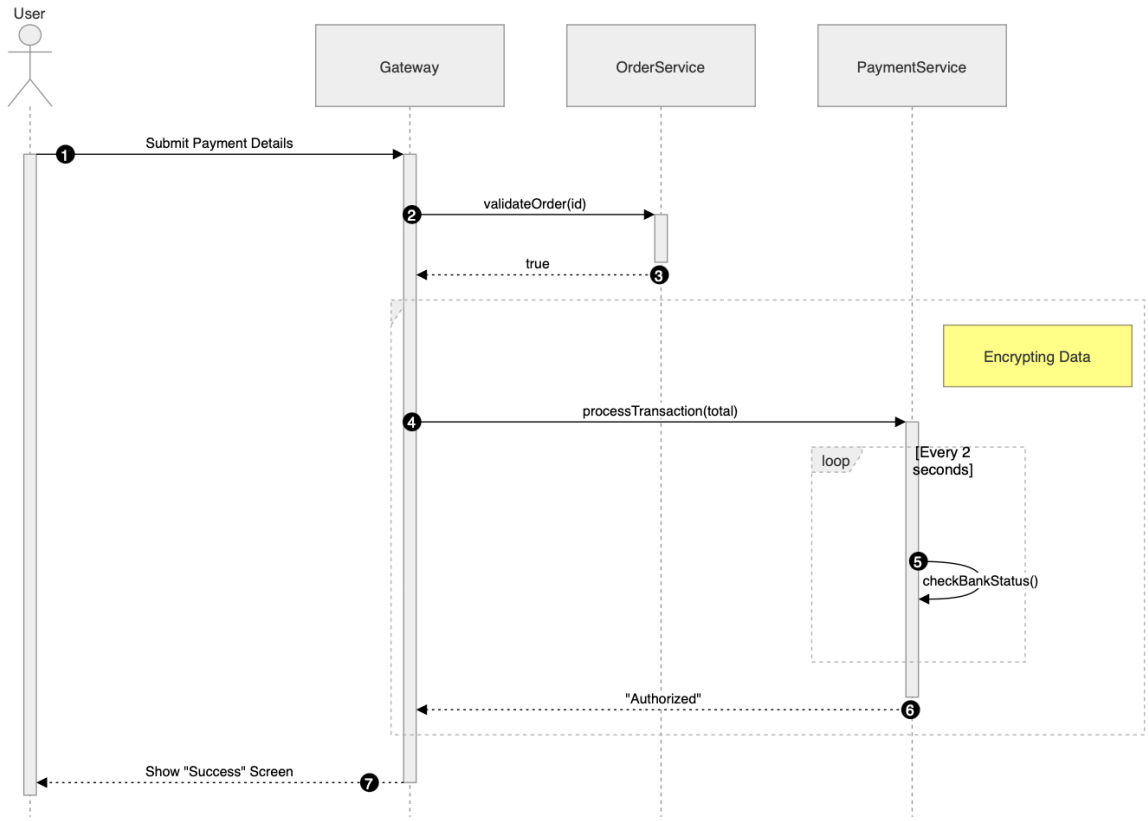


Advanced Sequence Diagram with Notes and Loops:

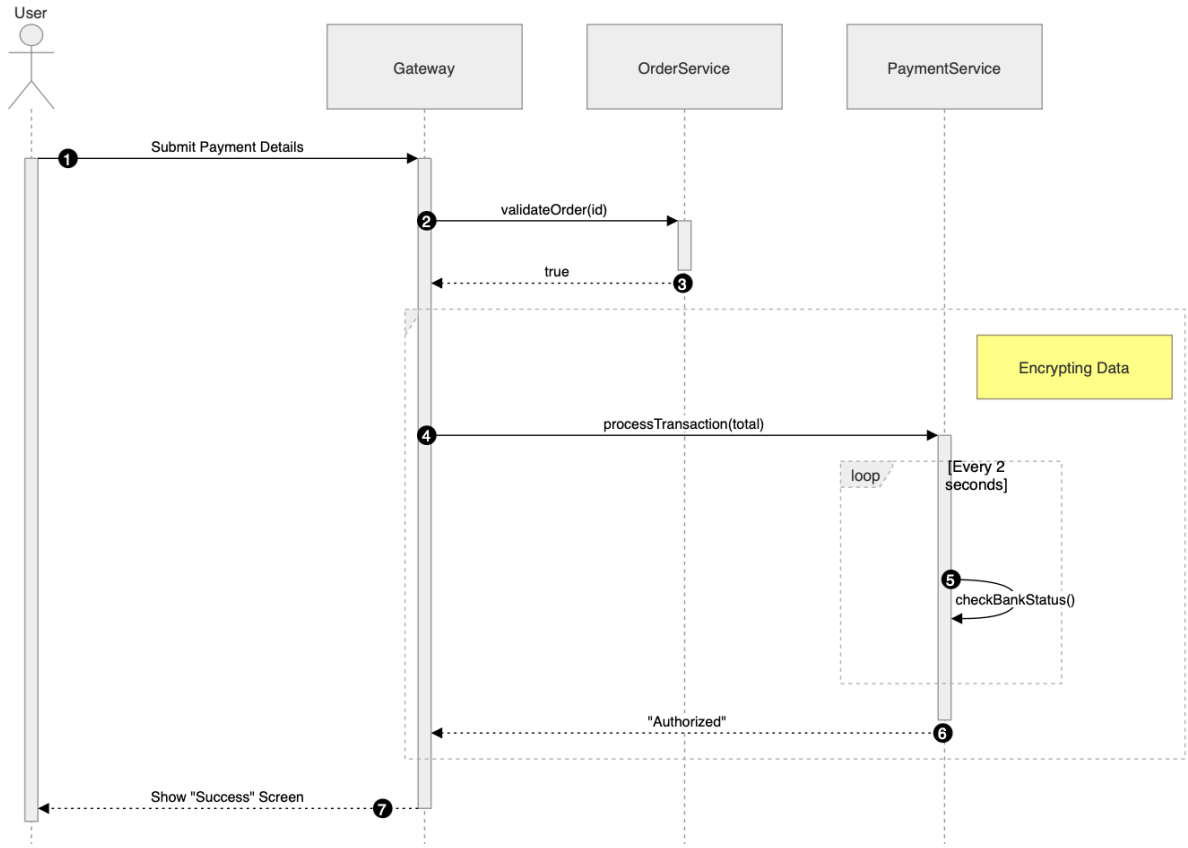
Code snippet

```
sequenceDiagram
    autonumber
    actor User
    participant Gateway
    participant Order as OrderService
    participant Pay as PaymentService

    User->>Gateway: Submit Payment Details
    activate User
    activate Gateway


    Gateway->>Order: validateOrder(id)
    activate Order
    Order-->>Gateway: true
    deactivate Order

    rect rgb(240, 240, 240)
        Note right of Pay: Encrypting Data
        Gateway->>Pay: processTransaction(total)
        activate Pay
        loop Every 2 seconds
            Pay->>Pay: checkBankStatus()
        end
        Pay-->>Gateway: "Authorized"
        deactivate Pay
    end

    Gateway-->>User: Show "Success" Screen
    deactivate Gateway
    deactivate User
```

Sequence Diagram (Low-Level Design)



Sequence Diagram (High-Level Design)



Student Practice Task

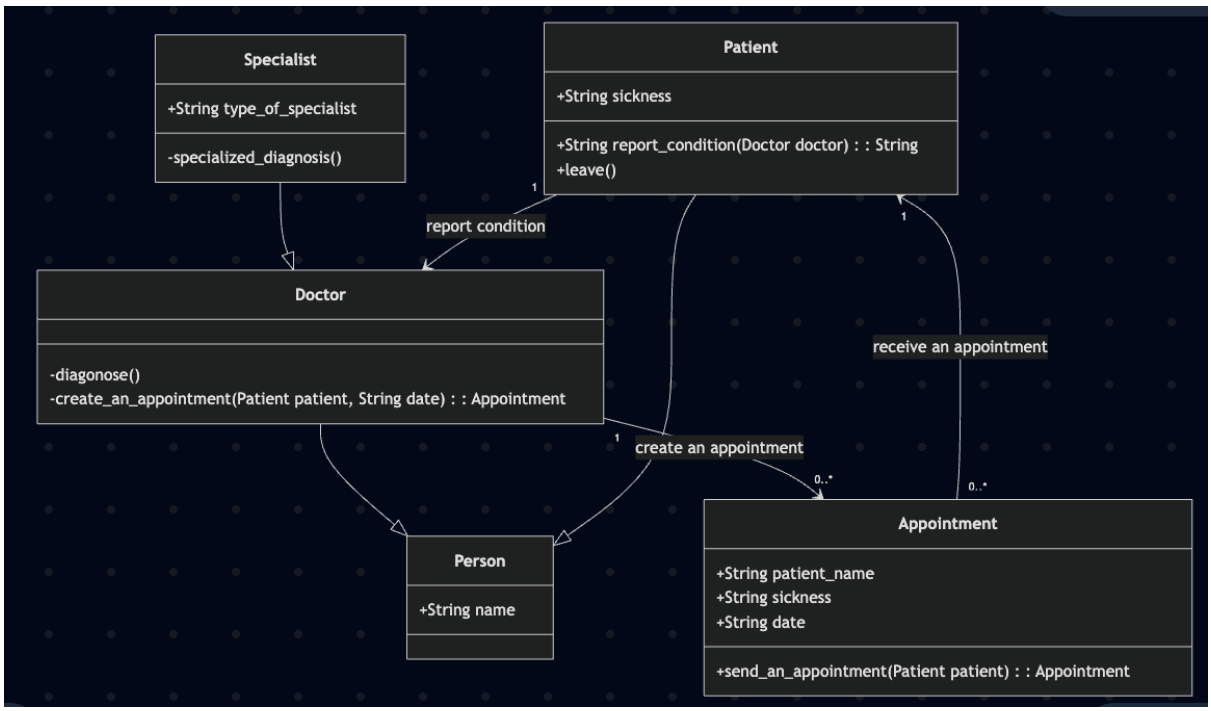
In-class Assignment:

Create a single Markdown file containing a **Class Diagram** and a **Sequence Diagram** for a **Hospital Management System**.

1. Class Diagram Requirements:
    - Classes for `Doctor`, `Patient`, and `Appointment`.
    - `Doctor` should have a method `diagnose()`.
    - Use inheritance if you add `Specialist` doctors.

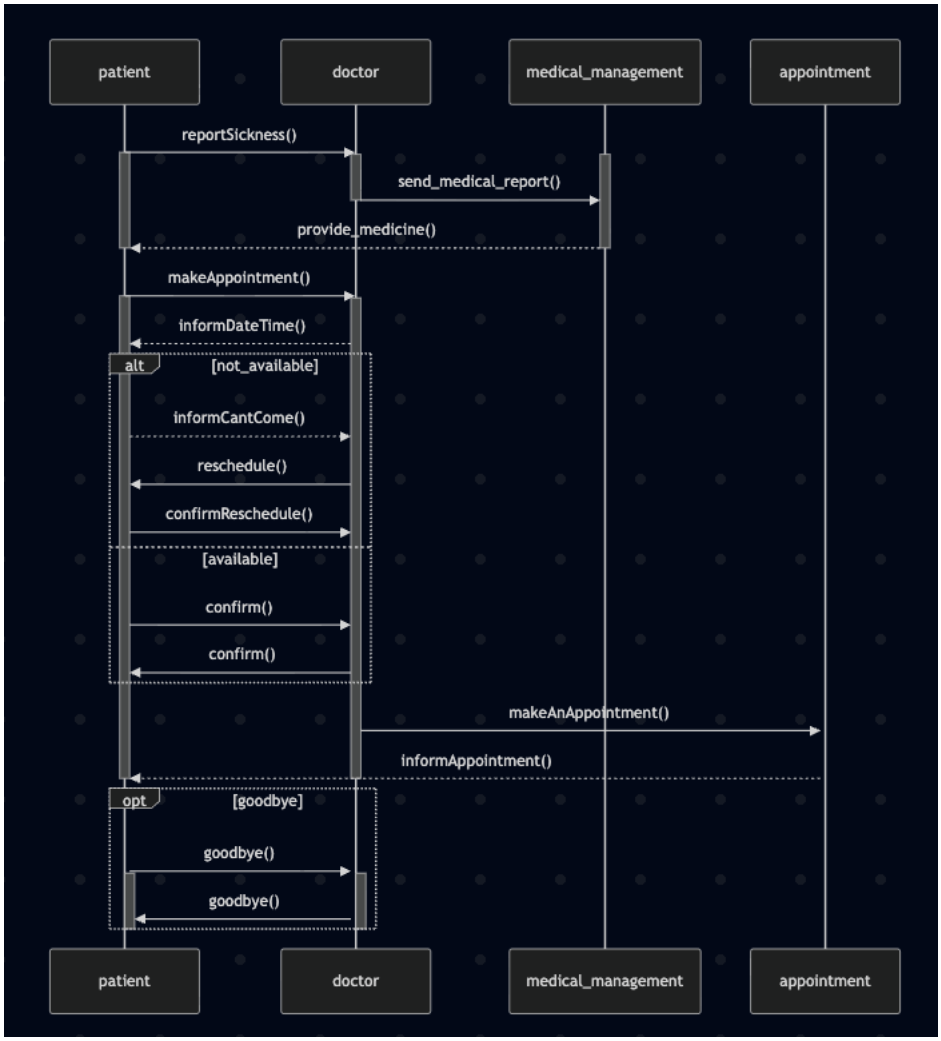
```
classDiagram

 class Person{
     +String name
 }

 class Doctor{
     -diagonose()
     -create_an_appointment(Patient patient, String date) : Appointment
     %%suppposed to have : Appointment appointment after this

}

class Patient{
    +String sickness
    +report_condition(Doctor doctor) : String
    +leave()
}

class Specialist{
    +String type_of_specialist
    -specialized_diagnosis()
}

class Appointment{
    +String patient_name
    +String sickness
    +String date
    +send_an_appointment(Patient patient) : Appointment

}

Doctor --|> Person
```

```
Patient --|> Person
Specialist --|> Doctor
Doctor "1" --> "0..*" Appointment : create an appointment
Patient "1" <-- "0..*" Appointment : receive an appointment
Patient "1" --> "1" Doctor : report condition
```




```
sequenceDiagram

    participant patient
    participant doctor
    participant medMan as medical_management
    participant appointment

    patient ->> doctor: reportSickness()
    activate patient
    activate doctor
    activate medMan
    doctor ->> medMan: send_medical_report()
    deactivate doctor
    medMan -->> patient : provide_medicine()

    deactivate patient
    deactivate medMan

    patient ->> doctor : makeAppointment()
    activate patient
```

```
    activate doctor
    activate appointment
    doctor -->> patient : informDateTime()
    alt not_available
        patient -->> doctor : informCantCome()
        doctor ->> patient : reschedule()
        patient ->> doctor : confirmReschedule()
    else available
        patient ->> doctor : confirm()
        doctor ->> patient : confirm()

    end
    doctor ->> appointment : makeAnAppointment()
    appointment -->> patient : informAppointment()

    deactivate patient
    deactivate doctor

    activate patient
    activate doctor
    Opt goodbye
        patient ->> doctor : goodbye()
        activate patient
    activate doctor
        doctor ->> patient : goodbye()

    end
    deactivate patient
    deactivate doctor
```



2. Sequence Diagram Requirements:
    - The `Patient` books an `Appointment`.
    - The `Patient` checks the `Doctor's` schedule.
    - Include an `alt` block for if the doctor is busy vs. available.

Submission: Export your diagrams as `.png` files from the Live Editor or submit the `.md` source code via the university portal.